

NWIS — ML Engineer Implementation Roadmap

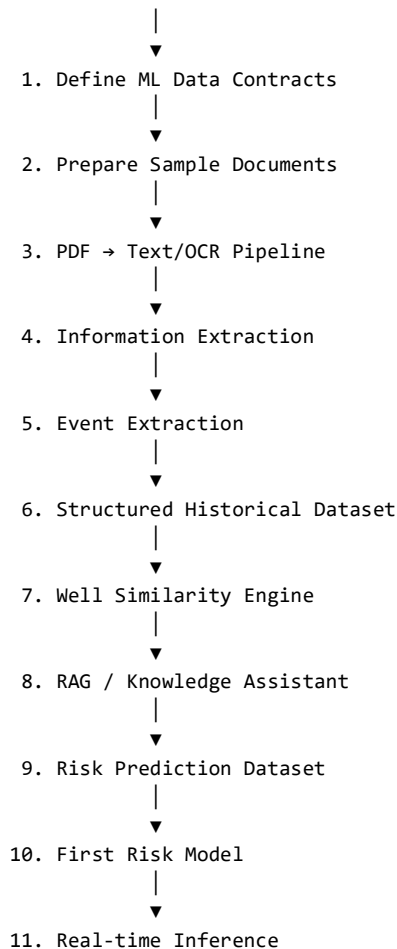
Nearby Wells Intelligence System | ML/AI Domain | Senior Team Lead Execution Plan

Purpose: A practical step-by-step plan for the ML engineer, including what can be developed independently, what must be coordinated with Backend, and the order in which the intelligence layer should be built.

1. Executive Decision: Do Not Wait for Backend

The ML engineer should start immediately. Backend and ML only need to agree on shared data/API contracts before integration. Backend implementation is not a prerequisite for most ML work.

YOUR ML TRACK



2. What the ML Engineer Owns

- Document AI: PDF classification, OCR, text and table extraction.
- NLP: entity extraction, event extraction, depth/formation association, cause and mitigation extraction.
- Historical event normalization and dataset creation.
- Well/formation/depth similarity engine.
- Embeddings, retrieval and RAG knowledge assistant.

- Feature engineering for drilling-risk models.
- Risk prediction models and evaluation.
- Model explainability, provenance and confidence.
- DVC/MLflow/reproducibility and model lifecycle.
- Stable ML input/output contracts for Backend integration.

3. ML Development Philosophy

- Do not start with the risk model. First build the institutional-memory data foundation.
- Develop against synthetic/representative data until authorized OIL data is available.
- Build reproducible pipelines rather than leaving logic inside notebooks.
- Keep every extracted fact traceable to its original document and page.
- Start with simple, strong baselines before advanced deep-learning models.
- Use Backend only as the integration layer; do not make the ML system depend on direct frontend access.
- Freeze schemas early so Backend can build against your outputs while ML continues independently.

4. Step 1 — Define ML Data Contracts

This is the first task. Before building extraction or models, define the structures that ML will produce and consume. Send these contracts to the Backend developer.

4.1 Drilling Event Schema

```
{
  "event_id": "EVT-001",
  "well_id": "OIL-102",
  "event_type": "MUD_LOSS",
  "depth_md": 2845,
  "depth_tvd": null,
  "formation": "Formation-X",
  "severity": "HIGH",
  "description": "Severe mud losses observed while drilling fractured limestone.",
  "cause": "Fractured formation",
  "mitigation": "LCM treatment",
  "outcome": "Losses reduced",
  "source_document": "WCR_OIL102.pdf",
  "source_page": 42,
  "confidence": 0.93
}
```

4.2 Well Schema

```
{
  "well_id": "OIL-102",
  "name": "OIL-102",
  "latitude": 26.123,
  "longitude": 94.123,
  "field": "FIELD-X"
}
```

4.3 Risk Prediction Schema

```
{
  "well_id": "OIL-102",
  "depth_md": 2848,
  "risk_type": "MUD_LOSS",
  "probability": 0.78,
  "level": "HIGH",
  "model_version": "mud-loss-v1"
}
```

4.4 RAG Response Schema

```
{
  "answer": "...",
  "sources": [
    {
      "document_id": "DOC-102",
      "page": 42
    }
  ]
}
```

Backend should design persistence and API responses around these contracts rather than inventing a different representation during implementation.

5. Step 2 — Build the ML Repository

```
ml/
├── README.md
├── requirements.txt
├── configs/
│   ├── extraction.yaml
│   ├── embeddings.yaml
│   ├── similarity.yaml
│   └── risk_models.yaml
├── data/
│   ├── raw/
│   ├── interim/
│   ├── processed/
│   └── external/
├── pipelines/
│   ├── ingest_documents.py
│   ├── ocr_pipeline.py
│   ├── extraction_pipeline.py
│   ├── embedding_pipeline.py
│   ├── feature_pipeline.py
│   └── training_pipeline.py
├── src/
│   ├── document_processing/
│   ├── ocr/
│   ├── nlp/
│   ├── extraction/
│   ├── embeddings/
│   ├── retrieval/
│   ├── similarity/
│   ├── features/
│   ├── risk/
│   └── evaluation/
├── models/
│   ├── trained/
│   └── artifacts/
├── notebooks/
├── tests/
├── dvc.yaml
└── params.yaml
```

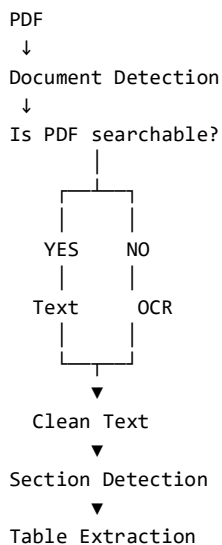
6. Step 3 — Prepare Sample Documents

For development, obtain authorized representative OIL documents where possible. If those are not yet available, create a synthetic dataset so the pipeline can be developed immediately.

- Well Completion Reports (WCRs).
- Daily Drilling Reports (DDRs).
- Mud logging reports.
- Drilling reports.
- Completion reports.
- Cementing reports.
- Well trajectory/survey records.
- Synthetic reports containing controlled examples of drilling events.

Create at least enough synthetic variation to test multiple wells, formations, depths, event types, severities, causes and mitigation actions.

7. Step 4 — Build the PDF Processing Pipeline



Suggested first modules:

```
ml/src/document_processing/  
├─ pdf_reader.py  
├─ pdf_classifier.py  
├─ ocr.py  
├─ text_cleaner.py  
└─ table_extractor.py
```

The first executable pipeline should accept a report and produce machine-readable text, tables and metadata.

```
python pipelines/ingest_documents.py
```

Input:

```
data/raw/WCR_OIL_102.pdf
```

Output:

```
data/processed/OIL_102/  
├─ text.json  
├─ tables.json  
└─ metadata.json
```

8. Step 5 — Build NLP Information Extraction

Convert extracted report text into normalized entities and events.

```
Raw Text
↓
Sentence Segmentation
↓
Entity Extraction
↓
Depth Extraction
↓
Formation Extraction
↓
Event Classification
↓
Cause Extraction
↓
Mitigation Extraction
↓
Structured Event
```

Example source sentence:

Severe mud losses were observed at 2845m while drilling fractured limestone.

Expected structured output:

```
{
  "event_type": "MUD_LOSS",
  "depth_md": 2845,
  "severity": "HIGH",
  "formation": "Limestone"
}
```

Start with a hybrid baseline: regex/rules for highly deterministic fields, NER/entity extraction for domain terms, event classification, and LLM-assisted extraction where it adds value. Keep the output schema deterministic and validated.

9. Step 6 — Define the Event Taxonomy

EVENT_TYPES

MUD_LOSS
KICK
STUCK_PIPE
FISHING
TORQUE_SPIKE
PRESSURE_SPIKE
OVERPRESSURE
LOST_CIRCULATION
CEMENTING_FAILURE
CASING_PROBLEM
NPT
FORMATION_CHANGE
WELL_CONTROL

SEVERITY

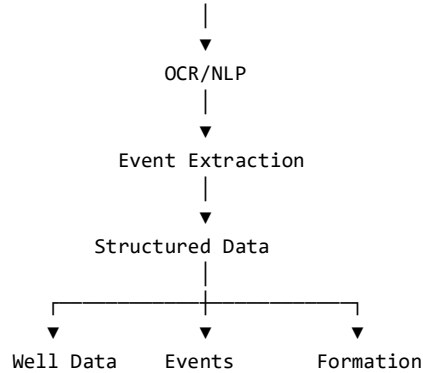
LOW
MEDIUM
HIGH
CRITICAL

This taxonomy must be shared with Backend and Frontend. It is the common vocabulary used by extraction, database storage, filters, risk models and dashboard alerts.

10. Step 7 — Build the Structured Historical Dataset

After PDF processing and extraction, create a reliable tabular/Parquet dataset.

Historical PDFs



well_id	depth	formation	event	severity	cause
OIL-001	2845	F-X	MUD_LOSS	HIGH	Fracture
OIL-001	3120	F-Y	STUCK_PIPE	HIGH	Differential
OIL-002	2870	F-X	MUD_LOSS	MEDIUM	Fracture
OIL-003	2910	F-X	TORQUE_SPIKE	MEDIUM	Formation

This dataset is the foundation for similarity, RAG metadata filters, feature engineering and predictive modeling.

11. Step 8 — Add Provenance and Evidence

Every extracted fact must retain enough information for an engineer to verify it.

- Document ID
- Page
- Section
- Original text
- Extraction model
- Extraction timestamp
- Confidence

If the system reports 'Mud loss at 2845 m', an engineer should be able to open the exact source document/page from which that fact came.

12. Step 9 — Build the Well Similarity Engine

This should be built before predictive risk modeling because it directly addresses the core nearby-well intelligence requirement.

- Active Well
 - ↓
 - Nearby wells
 - +
 - Same formation
 - +
 - Similar depth
 - +
 - Similar geological conditions

↓

Similarity calculation

Potential feature groups:

- Geographical proximity.
- Formation similarity.
- Depth/interval similarity.
- Reservoir/geological similarity.
- Trajectory similarity.
- Drilling-parameter similarity.
- Historical-event similarity.

Do not treat an arbitrary weighting formula as validated science. Start with an interpretable baseline and validate the ranking against domain-expert judgments and available outcomes.

```
{
  "active_well": "OIL-200",
  "similar_wells": [
    {
      "well_id": "OIL-102",
      "similarity": 0.92
    },
    {
      "well_id": "OIL-087",
      "similarity": 0.86
    }
  ]
}
```

13. Step 10 — Build the RAG Knowledge Assistant

Historical Documents

↓

Chunking

↓

Embeddings

↓

Vector Database

↓

Retrieval

↓

Relevant reports/events/wells

↓

LLM

↓

Evidence-backed answer

Use metadata filters such as field, well, formation, depth interval and event type. The assistant should return source document/page references with its answer.

Example question: 'What happened near 2800 m in wells within 5 km of the active well?' The system should retrieve relevant reports and events, then generate an answer grounded in those sources.

14. Step 11 — Build the Risk Prediction Dataset

Only begin this after the historical data foundation is working and there is sufficient labeled data.

Recommended first target: mud-loss prediction.

```
X =
depth
formation
ROP
WOB
RPM
torque
SPP
flow rate
mud weight
ECD
lithology
offset well events
distance
historical context
```

```
Y =
mud_loss_event
```

Create depth/time-aligned training examples. Be especially careful about leakage: rows from the same well should not be randomly scattered across train and validation sets in a way that lets the model memorize the well.

15. Step 12 — Train the First Risk Model

Start with interpretable tabular baselines.

- Logistic Regression baseline.
- Random Forest baseline.
- XGBoost or another strong gradient-boosted tree baseline.
- Compare performance and calibration.
- Use well-wise/time-wise splits where appropriate.
- Track precision, recall, PR-AUC/ROC-AUC, false-alert rate and calibration.
- Analyze performance separately across wells and formations when sample size allows.

Do not start with deep learning simply because the project is AI-enabled. The model should be chosen based on data characteristics, validation evidence and operational usefulness.

16. Step 13 — MLflow and DVC

Use DVC for reproducible datasets/pipelines and MLflow for experiments and model artifacts.

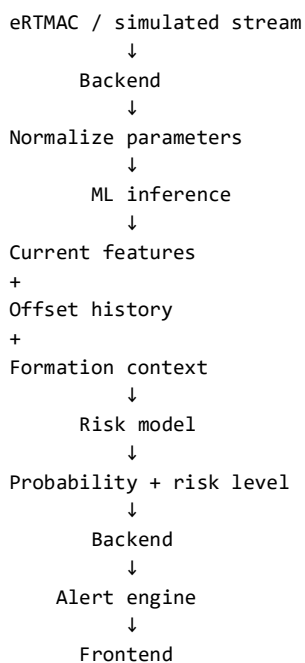
```
DVC
├─ raw/processed dataset versions
├─ pipeline stages
└─ reproducibility
```

```
MLflow
├─ parameters
├─ metrics
├─ artifacts
├─ model versions
└─ experiments
```

A model should not move to Backend integration until its training data, evaluation results, version and inference contract are documented.

17. Step 14 — Real-Time Inference

This is the final ML stage, after offline extraction, historical correlation and initial predictive models are stable.



Before connecting an actual OIL stream, build a simulated real-time stream. This lets the three teams validate the full system without waiting for production connectivity.

18. What You Can Do Without Backend

You can develop most of the initial ML system independently.

- PDF processing.
- OCR.
- Text cleaning.
- Table extraction.
- Entity extraction.
- Event extraction.
- Event taxonomy.
- Structured dataset generation.
- Provenance tracking.
- Similarity prototype.
- Embeddings and RAG prototype.
- Feature engineering.
- Offline risk-model experiments.
- DVC and MLflow pipelines.

Use JSON, CSV and Parquet as temporary interchange formats. Once Backend's database/API is ready, replace the temporary file boundary with the agreed API/data layer.

19. How to Coordinate With Backend

Send the Backend developer these four artifacts first:

1. Well schema.

- 2. Event schema.
- 3. Risk prediction schema.
- 4. RAG response schema.

Recommended message to Backend:

Build the backend around these contracts. I will develop the ML pipeline independently and we will integrate through these interfaces.

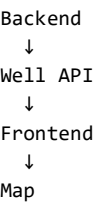
Backend should then create database tables and API schemas that preserve the same identifiers, units and enumerations.

20. Cross-Team Integration Rules

ML produces	Backend does	Frontend receives
Structured events	Validate + persist	Event timeline/details
Well similarity	Combine with GIS proximity	Nearby-well ranking
RAG answer + sources	Gateway/auth/audit	Answer + evidence
Risk prediction	Persist + alert policy	Risk card + alert
Model metadata	Track version	Display model/version context

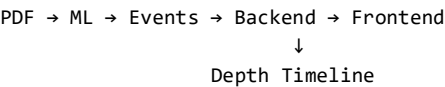
21. The First Three Integration Milestones

Milestone 1 — Active Well Intelligence



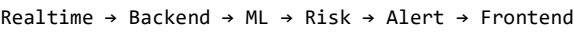
Deliverable: active well and nearby wells are visible on the map.

Milestone 2 — Historical Memory



Deliverable: historical drilling events from reports appear against the correct well/depth and can be traced to their sources.

Milestone 3 — Predictive NWIS



Deliverable: a simulated live drilling stream produces a risk prediction and an explainable alert in the dashboard.

22. Your Day 1 Execution Plan

- 5. Create the ML repository structure.
- 6. Create the event taxonomy.
- 7. Define Well, Event, Risk and RAG schemas.
- 8. Send those contracts to the Backend developer.
- 9. Collect authorized representative documents, or create synthetic reports.
- 10. Implement PDF text extraction.
- 11. Implement OCR fallback for scanned PDFs.
- 12. Save text/tables/metadata in processed output.

13. Create the first extraction prototype.
14. Produce the first structured event JSON.
15. Write tests around extraction of well ID, depth, formation and event type.

Day 1 success criterion: you can run one command that takes a sample PDF and produces structured, traceable output.

23. Day 1 Target

```
python pipelines/ingest_documents.py
```

INPUT

```
data/raw/WCR_OIL_102.pdf
```

OUTPUT

```
data/processed/OIL_102/
```

```
|— text.json
|— tables.json
|— metadata.json
|— events.json
```

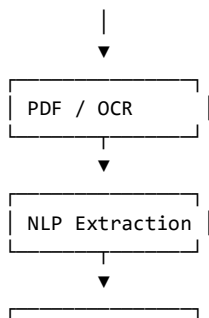
The first major ML milestone is NOT 'build an ML model'. It is: PDF → OCR/Text → NLP → Structured Drilling Events → Verified Dataset.

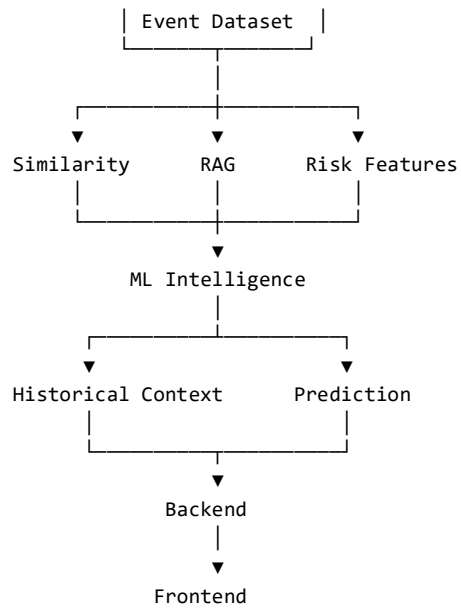
24. Recommended Immediate Sequence

16. Contracts and taxonomy.
17. Repository and environment.
18. Representative/synthetic documents.
19. PDF/OCR pipeline.
20. Text/table extraction.
21. NLP entity extraction.
22. Event extraction and normalization.
23. Provenance/evidence tracking.
24. Structured historical dataset.
25. Similarity engine.
26. RAG knowledge assistant.
27. Risk-model dataset.
28. Mud-loss baseline model.
29. MLflow/DVC model lifecycle.
30. Simulated real-time inference.
31. Backend integration.
32. Production data and eRTMAC integration after validation.

25. Final ML Architecture

OIL DOCUMENTS





26. Final Rule for the ML Engineer

Do not wait for Backend. Agree on contracts, then build the ML pipeline independently. Your first objective is to turn unstructured drilling history into a trustworthy, structured, traceable dataset. Once that foundation exists, the similarity engine, RAG assistant and predictive-risk layer can all be built on top of it.

Backend should be treated as the integration boundary, not as a prerequisite for ML development.